

LLM-Based Embodied Control with Code Generation and Self-Correction

Team 7

Yibai Sun, Peiming Chen, Rui He, Yinjie Liao, Jiaxi Zhong

AIR5021 Robot Manipulation

Abstract—This paper presents a natural-language robot control system for the Sagittarius 6-DOF real robotic arm based on Code as Policies and Language Model Programs (LMPs). The system uses an LMP reasoning layer to translate natural-language tasks into executable Python policy code. Through prompt engineering, few-shot code generation, and hierarchical function generation, the model performs object parsing, position parsing, spatial-relation reasoning, and multi-step task decomposition. The generated policy code does not directly control the hardware. Instead, it calls a unified set of high-level robot APIs, which are mapped by the LMP_wrapper to SagittariusEnv. The environment then grounds these actions to real-robot execution through ROS, MoveIt, and the SGRCtrl action server. The system supports tabletop pick-and-place, placement, arrangement, and stacking tasks. Unlike purely open-loop execution, the project implements perception-driven execution-level self-correction: when a pick failure is identified as `grasp_failed`, the robot returns to a canonical observation pose, refreshes camera observations, updates the `world_state`, and retries the grasp using the newly estimated object position. If the target object is not visible or repeated retries fail, the system returns a structured failure reason. The current self-correction mechanism is an execution-level closed-loop recovery strategy rather than a full LMP-level online code-rewriting repair loop.

I. Introduction

A central challenge in natural-language robot control is to reliably map human semantic intent to perception, planning, and physical action in the real world. Traditional tabletop manipulation systems often rely on fixed state machines, predefined trajectories, or manually designed task planners. Such methods can be stable in structured environments, but they have limited scalability when handling open-ended natural-language commands. For example, the instruction “move the rightmost block to the top region” requires the system to recognize objects, understand spatial relations, query current object coordinates, and invoke appropriate pick-and-place actions.

Recent large language models have shown strong capabilities in natural-language understanding, code generation, and task decomposition. Code as Policies (CaP) proposes an important paradigm: instead of producing only natural-language plans, large language models can directly generate executable robot policy code. The generated code can contain variables, conditional branches, loops, function calls, and numerical computation, making it more suitable than plain-text plans for connecting perception APIs and control APIs. CaP refers to such language-

model-generated executable programs as Language Model Programs (LMPs) [1].

Based on CaP, this project adapts LMPs to the Sagittarius real robotic arm platform. The goal is not to simply reproduce the tabletop prompt from the original CaP setting, but to build a complete pipeline from natural-language input, LMP code generation, Python code execution, environment wrapping, ROS actions, and real Sagittarius robot motion. The system also incorporates feedback-based recovery during real execution. In real manipulation, grasp-point error, object displacement, camera noise, empty grasps, and motion-planning failures can all cause open-loop code execution to fail. Therefore, this project extends “code as policy” into “code as feedback-aware executable policy.”

The main contributions of this project are summarized as follows:

- 1) We adapt Code as Policies to the Sagittarius real robotic arm and execute real pick-and-place tasks through a ROS action backend.
- 2) We build an end-to-end pipeline connecting natural language, LMP code generation, high-level APIs, environment wrapping, and real-robot execution.
- 3) We implement execution-level local self-correction after grasp failure, enabling the system to refresh perception and retry grasping based on updated object coordinates.

II. Related Work

A. Language-Grounded Robot Control

Enabling robots to understand human language is a long-standing problem in embodied AI and human-robot interaction. Earlier methods usually parse natural language into symbolic representations, temporal logic, or predefined action sequences, which are then executed by robot skills [2]. These methods are interpretable, but they often require substantial manual rules and task-specific design, limiting their generalization to open-ended language and new scenes.

With the development of large language models, researchers have started to use LLMs for robot task planning. SayCan combines the high-level planning ability of language models with robot affordance value functions, biasing the robot toward actions that are both semantically appropriate and physically feasible [3]. Inner Monologue

further converts environment feedback, success-detector outputs, and vision-language model results into language context for online robot planning [4]. Socratic Models demonstrates compositional reasoning by combining pre-trained vision, language, and action models [5]. These works show that LLMs can serve as high-level task planners connecting language and robot actions, but their outputs are often action sequences or natural-language plans rather than directly executable policy code.

B. Code as Policies and Language Model Programs

Code as Policies uses LLM-generated code as robot policy and emphasizes the expressiveness of code compared with natural-language plans. Code can directly call perception APIs and control APIs, use conditional statements, loops, variables, and external libraries for spatial-geometric computation, and construct more complex policy logic through hierarchical function generation [1]. Compared with language planners that output only action sequences, CaP has the advantage that its output is itself an executable program, which can naturally express feedback loops, geometric reasoning, and multi-step compositional behaviors.

ProgPrompt also uses programmatic prompts for robot task planning, where Python-like structures help LLMs generate executable action sequences [6]. However, ProgPrompt focuses more on representing high-level plans with program structure, while CaP emphasizes treating generated code as robot policy and grounding it through perception and control APIs. This project follows the LMP idea from CaP, but focuses on engineering the full Sagittarius real-robot execution pipeline and implementing execution-level recovery after grasp failure.

C. Execution Feedback and Robot Self-Correction

Real-robot execution is inevitably affected by perception noise, object displacement, control error, and environmental disturbances. Therefore, feeding execution feedback back into planning or control is important for improving system robustness. DoReMi studies plan-execution misalignment detection and recovery, enabling the system to detect deviations between language-model-generated plans and actual execution [7]. CoPAL uses LLMs for corrective planning by using execution feedback to revise robot action plans [8]. Code-as-Monitor explores the use of code to express visual monitoring and constraint checking, enabling robotic failure detection and risk monitoring [9].

These works share a common goal with our project: robots should not merely execute one-shot open-loop plans, but should also recover from failures. The difference is that our current implementation realizes local execution-level self-correction rather than full LMP-level replanning. Specifically, when Sagittarius returns `grasp_failed` during a `PICK_XYZ` action, the system does not immediately ask the LMP to regenerate code.

Instead, `SagittariusEnv.step()` internally moves the arm back to an observation pose, refreshes perception, updates the `world_state`, and retries the grasp using the updated coordinates. This design is suitable for low-level grasp errors because it is fast, explicit, and does not require another LLM call.

D. ROS and Motion Planning Middleware

The real-robot execution layer of this project is built on ROS and MoveIt. ROS provides message communication, node organization, and action-server mechanisms commonly used in robot systems [10]. MoveIt provides motion planning, collision checking, inverse kinematics, and trajectory execution for robotic arms [11]. In this system, LMP-generated high-level actions do not directly operate ROS topics or MoveIt interfaces. Instead, they are wrapped by SagittariusEnv into `SGRCtrlGoal` messages and sent to the `/sgr532/sgr_ctrl` action server. This layered design allows the language model to focus on task logic, while low-level motion execution, visual perception, and failure states are encapsulated inside the robot environment layer.

III. System Overview

The system consists of an LMP reasoning layer, a code execution layer, an environment wrapping layer, and a ROS-based real-robot execution layer. Overall, user natural language first enters the LMP reasoning layer. The system generates Python policy code based on prompts, few-shot examples, the current `world_state`, and available APIs. The generated code calls high-level robot APIs through the `LMP_wrapper`. Finally, SagittariusEnv converts semantic actions into perception updates, coordinate parsing, action packaging, and ROS action calls. The overall architecture is shown in Fig. 1.

Fig. 1 illustrates the main extension of this project beyond a direct CaP reproduction: the system not only generates robot code using LMPs, but also grounds the generated code into the Sagittarius real-robot execution pipeline and adds an execution-feedback-driven local recovery mechanism. The current self-correction is not a full LMP online code-rewriting repair process; it is implemented as an execution-level closed-loop recovery mechanism inside `SagittariusEnv.step()`.

IV. Language Model Program Design

A. LMP Reasoning Layer

The LMP reasoning layer is the core module responsible for translating language into code. It maintains prompt templates, few-shot examples, sub-LMPs, a function generator, and a code execution environment. A typical LMP call includes constructing the prompt, appending the current object list as context, calling the code generation model, obtaining a Python code string, using `LMPFGen` to complete undefined helper functions, and executing the generated code through `exec_safe` within a controlled

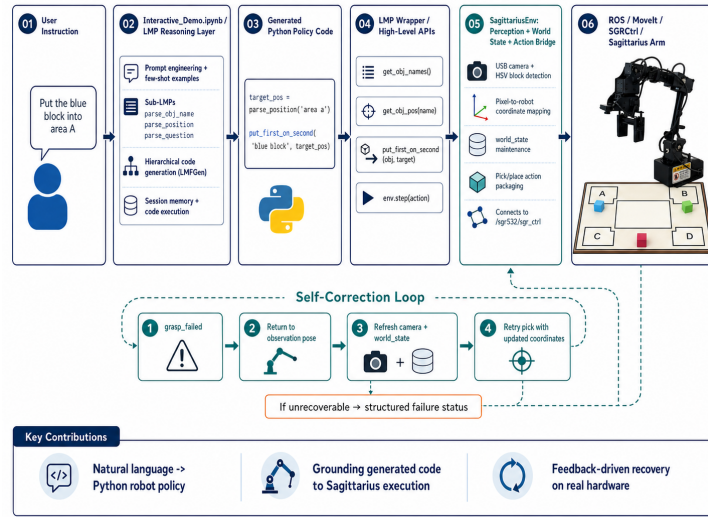


Fig. 1. System overview of the proposed LMP-based Sagittarius real-robot control framework. The pipeline starts from user natural-language instructions, generates Python policy code through the LMP reasoning layer, maps high-level APIs to SagittariusEnv, and finally executes actions through ROS, MoveIt, SGRCtrl, and the Sagittarius arm. The lower part illustrates the execution-level self-correction loop.

namespace. In implementation, this module is instantiated through an interactive Python execution interface; however, this paper describes it at the system level rather than referring to it as a specific notebook file.

The system uses multiple cooperating sub-LMPs rather than relying on a single model to perform all reasoning in one step. The top-level tabletop_ui converts complete user commands into policy code. `parse_obj_name` maps descriptions such as “the blocks” or “rightmost block” to concrete object names or object lists. `parse_position` maps spatial descriptions such as “area A” or “top side” to coordinates. `parse_question` answers scene-relation questions. LMPGen recursively generates helper functions when the generated code refers to undefined functions. This structure follows the idea of hierarchical code generation in CaP, allowing the LMP to first express high-level logic and then incrementally complete helper functions.

B. Prompt Engineering and Few-Shot Examples

The prompt design in this project follows a Hints + Few-shot Examples structure. Hints tell the model which first-party APIs are available, such as `get_obj_pos`, `get_obj_names`, and `put_first_on_second`. Few-shot examples show how natural-language instructions should be translated into Python policy code. In this way, the LMP learns that object descriptions should be handled by `parse_obj_name`, position descriptions should be handled by `parse_position`, and final robot actions should be executed through `put_first_on_second`.

For example, for a simple command such as “put the blue block into area A”, the LMP can generate the following code:

```
# put the blue block into area A.
target_pos = parse_position(
    'area A',
    f'objects = {get_obj_names()}'
)
put_first_on_second('blue block', target_pos)
```

For spatial-relation tasks, the LMP generates code containing numerical comparison and loops. For example, in “move the block closest to the right side to the top side”, the system first enumerates all current blocks, queries the position of each block, and then selects the target object according to its distance from the right side. This type of reasoning does not rely only on the language model’s internal knowledge. Instead, it is explicitly performed through generated Python and NumPy-style geometric computation.

C. High-Level Robot APIs

When generating code, the LMP does not directly handle ROS actions, MoveIt goals, or low-level robot-control messages. It only sees a set of semantically clear high-level functions:

```
get_obj_names()
get_obj_pos(name)
parse_position(...)
put_first_on_second(obj,target)
```

This interface design decouples task logic from low-level robot communication, allowing the LMP to focus on object selection, spatial relations, and action composition. For arrangement tasks, the LMP does not need to call new low-level control interfaces. Instead, it can generate a loop structure: first using `parse_obj_name` to obtain the current list of blocks, then using `parse_position` or a geometric helper function to generate a list of target points, and finally repeatedly calling `put_first_on_second` to

complete multiple pick-and-place actions. This shows that the system’s extensibility comes mainly from high-level code generation and API composition rather than increasing the number of low-level control commands.

V. Real-Robot Grounding and Execution

A. SagittariusEnv Execution Layer

SagittariusEnv is the bridge between LMP policy code and the Sagittarius ROS backend. It connects to the /sgr532/sgr_ctrl action server, reads images from /usb_cam/image_raw, detects red, green, and blue blocks using OpenCV/HSV color segmentation, maps pixel coordinates to robot coordinates, and maintains the world_state of currently visible objects.

B. Perception and World State

The visual perception pipeline first converts camera images into HSV space. It then generates masks for different color thresholds, extracts contours, and computes object centers. Once a block is detected, the system uses a linear mapping from pixel coordinates to robot coordinates:

$$x_{\text{robot}} = k_1 y_{\text{pixel}} + b_1, \quad y_{\text{robot}} = k_2 x_{\text{pixel}} + b_2. \quad (1)$$

The world_state stores a mapping from object names to three-dimensional coordinates, such as the estimated positions of blue block, red block, and green block. The system also provides fixed named positions, including area a/b/c/d, top side, middle, right side, and corner points, so that natural-language expressions such as “area A” or “top right corner” can be grounded to robot coordinates.

Fig. 2 shows two examples of environment initialization and object-list refresh in the real-robot execution interface. The system first connects to the /sgr532/sgr_ctrl action server, moves the robot to safe scanning poses, scans the tabletop workspace, and finally writes detected colored blocks into env.object_list and world_state.

It should be noted that the current Sagittarius real-robot grounding mainly supports colored blocks and fixed tabletop areas. Although the system follows the tabletop prompt style of CaP, the current real-robot environment should not be described as fully supporting all object categories such as bowls, fruits, or plates.

C. Action Execution

env.step(action) is the core function for pick-and-place execution. Its input contains pick, place, object_name, and target_name. The function first normalizes coordinates and checks workspace boundaries:

$$x \in [0.00, 0.35], \quad y \in [-0.40, 0.40], \quad z \in [0.00, 0.30]. \quad (2)$$

If the pick or place coordinate is outside the workspace, the system directly returns pick_out_of_workspace or place_out_of_workspace. If the coordinates are valid, the system constructs a PICK_XYZ goal and sends it to the SGRCtrl action server. After a successful pick,

it constructs a PUT_XYZ goal to complete placement and updates the world_state. Therefore, high-level actions generated by the LMP are grounded into real-robot action-server requests and execution-result feedback.

VI. Execution-Level Self-Correction

In this paper, self-correction refers to an execution-level, perception-driven closed-loop recovery mechanism. It is not a full LMP online code-rewriting repair loop. The current system does not automatically feed every error back to the LMP and ask it to generate new repair code. Instead, recovery is implemented inside SagittariusEnv.step(). This design targets the frequent low-level error of grasp failure and enables faster and more stable local recovery.

When PICK_XYZ returns failure, the system first checks the failure reason. If the reason is not grasp_failed, the system directly returns structured failure information. If the reason is grasp_failed and retries remain, the system performs strict perception refresh: it moves to a canonical observation pose, waits for stabilization, reads multiple image frames, re-detects objects, and replaces the current world_state with the refreshed observation. If the target object is still visible, the system retries the pick using the new coordinate. If the target is not visible, it stops and returns object_not_visible_after_pick_fail.

The following pseudocode shows the core self-correction loop. It corresponds to the grasp-failure recovery path inside SagittariusEnv.step(), omitting ROS result fields, logs, and some engineering-specific exception branches.

```

pick_result = send_pick_goal(pick_xyz)

if pick_result.success:
    send_place_goal(place_xyz)

elif pick_result.reason == 'grasp_failed':
    refreshed_state = refresh_world_state_strict()

    if object_name in refreshed_state:
        pick_xyz = refreshed_state[object_name]
        retry_pick_goal(pick_xyz)
    else:
        return failure('object_not_visible_after_pick_fail')

else:
    return failure('pick_failed', pick_result.reason)

```

Fig. 3 shows a real execution log of stacking failure recovery. The LMP generates put_first_on_second(‘green block’, ‘red block’) according to the user instruction. After the execution layer detects GRASP_FAILED, it does not immediately terminate the task. Instead, it refreshes the world_state, reads the updated coordinate of the green block, and retries the grasp. This figure only shows log evidence of self-correction and does not interpret a generic robot-grasping image as a failure-recovery state.

This logic defines the boundary of the current self-correction implementation. The system does not call the LMP again to generate repair code after failure. Instead, it detects grasp_failed at the execution layer, returns to

```

# ... SAGITTARIUS REAL ROBOT SETUP ...
import sys
sys.path.append('.')
from sagittarius_env import SagittariusEnv

env = SagittariusEnv(arm_name='sgr532')

SAFE_SCAN_RESET_POSE_CANDIDATES = [
    (0.25, 0.00, 0.10),
    (0.25, 0.00, 0.12),
    (0.25, 0.00, 0.15),
]

MAP_SCAN_POSES = [
    (0.25, 0.00, 0.10),
    (0.25, 0.00, 0.10),
    (0.25, 0.00, 0.10),
]

def reset_arm_before_map_scan():
    for pose in SAFE_SCAN_RESET_POSE_CANDIDATES:
        if env.move(pose):
            return True
    try:
        cur = env.get_arm_pos().tolist()
    except Exception:
        cur = 'unknown'
    print(f'[WARN] reset_arm_before_map_scan failed. Continue with current pose: {cur}')
    return False

reset_arm_before_map_scan()
env.scan_world_state()
obj_list = env.object_list
lmp_tabletop_ui = setup_LMP(env, cfg.tabletop)
print('available objects:')
print(obj_list)

[INFO] [177538970.388083]: Waiting for Sagittarius Control Action Server: /sgr532/sgr_ctrl
[INFO] [177538970.388083]: Connected to Sagittarius control action server.
available objects:
['green block', 'red block']

```

(a) Initial scan with three blocks

```

# ... SAGITTARIUS REAL ROBOT SETUP ...
import sys
sys.path.append('.')
from sagittarius_env import SagittariusEnv

env = SagittariusEnv(arm_name='sgr532')

SAFE_SCAN_RESET_POSE_CANDIDATES = [
    (0.25, 0.00, 0.10),
    (0.25, 0.00, 0.12),
    (0.25, 0.00, 0.15),
]

MAP_SCAN_POSES = [
    (0.25, 0.00, 0.10),
    (0.25, 0.00, 0.10),
    (0.25, 0.00, 0.10),
]

def reset_arm_before_map_scan():
    for pose in SAFE_SCAN_RESET_POSE_CANDIDATES:
        if env.move(pose):
            return True
    try:
        cur = env.get_arm_pos().tolist()
    except Exception:
        cur = 'unknown'
    print(f'[WARN] reset_arm_before_map_scan failed. Continue with current pose: {cur}')
    return False

reset_arm_before_map_scan()
env.scan_world_state()
obj_list = env.object_list
lmp_tabletop_ui = setup_LMP(env, cfg.tabletop)
print('available objects:')
print(obj_list)

[INFO] [177538970.407081]: Waiting for Sagittarius Control Action Server: /sgr532/sgr_ctrl
[INFO] [177538970.407081]: Connected to Sagittarius control action server.
available objects:
['blue block', 'green block', 'red block']

```

(b) Scan under another tabletop state

Fig. 2. Real-robot world-state scanning on the Sagittarius platform. The system initializes SagittariusEnv, connects to the /sgr532/sgr_ctrl action server, moves the arm to safe observation poses, scans the workspace, and updates the visible object list.

```

#User input
user_input = 'stack the green block onto red block'
env.refresh_world_state(force=True)

print('Running policy...')
lmp_tabletop_ui(user_input, f'objects = {env.object_list}')

Running policy...
LMP tabletop_ui exec:

objects = ['green block', 'red block']
# stack the green block onto red block.
say('Stacking the green block onto the red block')
put_first_on_second('green block', 'red block')

robot says: Stacking the green block onto the red block
[INFO] [177538966.697115]: Adjusted place height for stacking: object-green block target-red block place_z=0.0608
[INFO] [177538966.708975]: Executing action via /sgr532/sgr_ctrl | object-green block target-red block pick=[0.2525201 0.08530477 0.02] place=[ 0.3087218 -0.09244851 0.06]

[WARN] [177538881.495202]: PICK_XYZ attempt 1/3 failed with GRASP_FAILED.
[WARN] [177538881.497698]: Pick attempt 1/3 grasp failed for 'green block'. Refreshing world state before retry.
[INFO] [177538885.845283]: Retrying pick for 'green block' at refreshed position [0.2692258734703064, 0.09849496930837631, 0.0199999999552965164].
[WARN] [177538902.486991]: PICK_XYZ attempt 2/3 failed with GRASP_FAILED.
[WARN] [177538902.489494]: Pick attempt 2/3 grasp failed for 'green block'. Refreshing world state before retry.
[INFO] [177538906.972813]: Retrying pick for 'green block' at refreshed position [0.19857892394065857, -0.019509585574269295, 0.0199999999552965164].

```

Fig. 3. Execution-level self-correction during a stacking task. The LMP generates a direct stacking policy, while the execution layer detects GRASP_FAILED, refreshes the world state, obtains an updated object position, and retries the pick action.

an observation pose, refreshes perception and world_state, and retries the grasp using the updated object coordinate. If the target object is not visible after the refresh, the system returns a structured failure reason rather than blindly continuing to execute the place action. A more complete LMP-level repair loop, where ROS, MoveIt, or visual errors are converted into semantic feedback and sent back to the LMP to generate repair patches, remains future work.

VII. Experiments and Demonstrations

A. LMP-based PyBullet Simulation

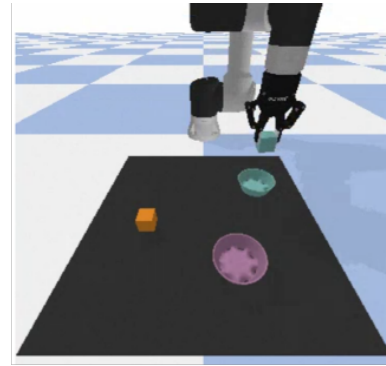
Before real-robot experiments, we build an LMP-based PyBullet simulation environment to verify whether the natural-language-to-policy-code generation logic is correct. PyBullet is a Python module for robotics, physics simulation, and machine learning. It supports loading articulated bodies from URDF, SDF, MJCF, and other formats, and provides forward dynamics, inverse dynamics, forward kinematics, inverse kinematics, collision detection, and ray-intersection queries [12]. Therefore, it is suitable as a lightweight validation platform before deploying policies to the Sagittarius real robot.

The core logic of this simulation environment is consistent with the real-robot LMP system. After receiving a natural-language instruction, the LMP reasoning layer still generates Python policy code based on prompts, few-shot examples, and the current object list. The generated code still calls the same high-level APIs, such as get_obj_names(), get_obj_pos(), parse_position(), and put_first_on_second(). The difference is that the backend during simulation is not SagittariusEnv or the ROS action server, but a PyBullet environment wrapper. This wrapper maps high-level pick-and-place actions to simulated end-effector motion, gripper closing, and object-pose updates.

This design keeps the simulation logic consistent with the real-robot logic at the LMP level. The LMP does not need to know whether the current execution backend is simulation or the real robot. It only needs to generate policy code in the same form. The backend environment is responsible for grounding high-level actions to different execution platforms. The PyBullet simulation is mainly used to validate object parsing, position parsing, multi-step task decomposition, and action-sequence correctness.



(a) Object recognition and scene initialization



(b) LMP policy execution in PyBullet

Fig. 4. LMP-based PyBullet simulation. (a) The simulation environment initializes a tabletop scene and prints the available objects, including colored blocks and bowls. (b) The simulated robot executes an LMP-generated pick-and-place policy using the same high-level API structure as the real-robot system.

TABLE I
Demonstration tasks used in the project

Demonstration	Capability shown
Move a block across all defined areas	Multi-step task decomposition and repeated API calls
Arrange blocks along a horizontal line	Spatial parsing and geometric target generation
Move the block left of the red block to area D	Relative-position reasoning
Question-driven block movement	Scene question answering combined with manipulation
Stacking with grasp retry	Execution-level self-correction

B. Demonstration Tasks

The current project uses qualitative demonstrations rather than large-scale quantitative benchmarking as its main experimental form. The demonstration tasks cover four capabilities: natural-language-to-policy-code generation, spatial-relation reasoning, multi-step action composition, and execution-level recovery from grasp failure. Table I summarizes the main task types shown in this report.

C. Real-Robot Policy Execution Demonstrations

To show the correspondence between LMP-generated code and real robotic-arm behavior, this section combines policy-code logs with real robot operation images. In each group, the code or log generated by the LMP is paired with a Sagittarius real-robot execution image from the corresponding task.

Fig. 5 shows an area-based movement task. The LMP first calls `parse_position('all defined areas')` to obtain coordinates for multiple areas, and then repeatedly calls `put_first_on_second` on the same object in a loop. The real-robot image shows the arm grasping the blue block and executing movement on the labeled area map.

Fig. 6 shows a line-arrangement task. The LMP first parses the list of currently visible blocks, then generates multiple target points according to the instruction “20cm

horizontal line near the top side with 3 points,” and places different blocks at these points. The real-robot image shows the arm operating in a multi-block scene during the arrangement process.

Fig. 7 shows a relative-position reasoning task. The user asks the system to “pick up the block on the left of red block and move it to area d.” The LMP queries block positions through `parse_obj_name` and infers the target object based on its relative position to the red block. The real-robot image shows the robot moving the inferred target block toward area D.

Fig. 8 shows a question-driven multi-step manipulation task. The system first answers “which block is closest to the right side” and moves the selected object. It then answers “which block is closest to the left side” and moves the selected object to the top-right corner. This example demonstrates the LMP’s ability to combine scene question answering, spatial computation, and sequential action generation.

VIII. Limitations and Future Work

The current system still has several limitations. First, the system inherits some colored bowl examples from the original CaP tabletop prompt. However, the current Sagittarius real-robot grounding mainly supports red, green, and blue blocks, fixed regions area A/B/C/D, and tabletop directional points. Future work should further align the prompt with the real environment, remove or reduce examples involving unsupported objects, and add more expressions corresponding to the objects and regions currently supported by the Sagittarius platform.

Second, the visual perception module is based on HSV thresholds and contour detection. Its advantage is simplicity and low latency, but it is sensitive to illumination, shadows, camera viewpoint, and background color. The linear mapping from pixel coordinates to robot coordinates also depends on calibration quality. Future work could introduce AprilTag or ArUco markers, depth cameras, or learning-based object detection, together with automatic

```

[1] #User input
user_input = 'pick up the blue block and move it across all defined areas'
env.refresh_world_state(force=True)

print('Running policy...')
tap_tabletop_ui(user_input, f'objects = (env.object_list)')

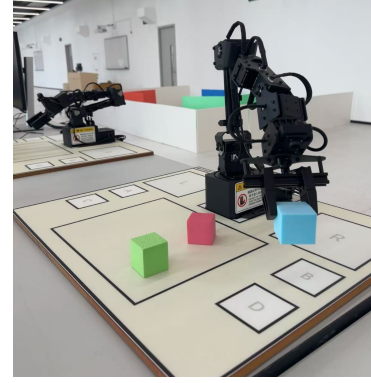
Running policy...
LMP tabletop_ui exec:
objects = ['blue block', 'green block', 'red block']
# pick up the blue block and move it across all defined areas
say('Sure - picking up the blue block and moving it across all defined areas')
areas = parse_position('all defined areas')
for area in areas:
    put_first_on_second('blue block', area)

robot says: Sure - picking up the blue block and moving it across all defined areas
LMP parse_position exec:
# all defined areas
ret_val = [get_obj_pos('area a'), get_obj_pos('zone b'), get_obj_pos('region c'), get_obj_pos('map area d')]

[INFO] [1777537856.559425]: Executing action via /sgr532/sgr_ctrl | object=blue block target=None pick=[ 0.2701345 -0.03394903 0.02 ]
place=[ 0.16 -0.25 0.02]
[INFO] [1777537889.034061]: Executing action via /sgr532/sgr_ctrl | object=blue block target=None pick=[ 0.16 -0.25 0.02] place=[0.16 0.25 0.02]
[INFO] [1777537924.229523]: Executing action via /sgr532/sgr_ctrl | object=blue block target=None pick=[0.16 0.25 0.02] place=[ 0.26 -0.25 0.02]
[INFO] [1777537961.0758071]: Executing action via /sgr532/sgr_ctrl | object=blue block target=None pick=[ 0.26 -0.25 0.02] place=[0.26 0.25 0.02]

```

(a) Generated policy for moving the blue block across defined areas



(b) Real robot execution for blue-block area-based movement

Fig. 5. Area-based movement demonstration. The user asks the system to “pick up the blue block and move it across all defined areas.” The LMP decomposes the command into a loop over named areas, while the real robot executes the corresponding pick-and-place actions with the blue block on the physical workspace.

```

[1] #User input
user_input = 'a 20cm horizontal line near the top side with 3 points'
env.refresh_world_state(force=True)

print('Running policy...')
tap_tabletop_ui(user_input, f'objects = (env.object_list)')

Running policy...
LMP tabletop_ui exec:
objects = ['blue block', 'green block', 'red block']
# a 20cm horizontal line near the top side with 3 points
say('Ok - arranging the blocks in a 20cm horizontal line near the top side')
block_names = parse_obj_names('the blocks', f'objects = {get_obj_names()}', f'objects = {get_obj_names()}', f'objects = {get_obj_names()}')
line_pts = parse_position('a 20cm horizontal line near the top side with {len(block_names)} points')
for block_name, pt in zip(block_names, line_pts):
    put_first_on_second(block_name, pt)

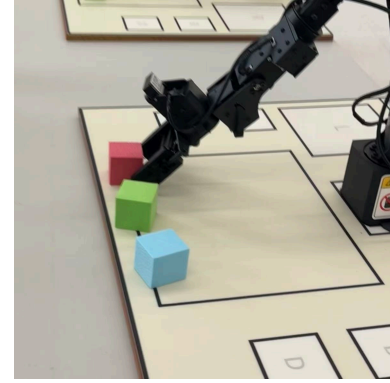
robot says: Ok - arranging the blocks in a 20cm horizontal line near the top side
LMP parse_obj_names exec:
objects = ['blue block', 'green block', 'red block']
# the blocks
ret_val = ['blue block', 'green block', 'red block']

LMP parse_position exec:
# a 20cm horizontal line near the top side with 3 points
middle_pos = get_obj_pos('top side')
start_pos = middle_pos + [0, 0, 0.2]
end_pos = middle_pos + [0, 0, 0.2]
line = make_line(start_pos, end_pos)
points = interpolate_pts_on_line(line=line, n=3)
ret_val = points

[INFO] [177754009.264281]: Executing action via /sgr532/sgr_ctrl | object=blue block target=None pick=[ 0.19171497 -0.08425342 0.02 ]
place=[0.35 0.2 0.02]
[INFO] [177754030.208622]: Executing action via /sgr532/sgr_ctrl | object=green block target=None pick=[0.20563948 0.10520959 0.02 ]
place=[0.35 0.2 0.02]

```

(a) Generated policy for arranging blocks along a horizontal line



(b) Real robot scene for multi-block arrangement

Fig. 6. Line arrangement demonstration. The LMP parses the required geometric layout, generates target points, and sequentially places visible blocks onto the computed positions. The real-robot image shows the arm operating in a multi-block scene consistent with the arrangement task.

```

[12] #User input
user_input = 'pick up the block on the left of red block and move it to area d'
env.refresh_world_state(force=True)

print('Running policy...')
tap_tabletop_ui(user_input, f'objects = (env.object_list)')

Running policy...
LMP tabletop_ui exec:
objects = ['blue block', 'green block', 'red block']
# pick up the block on the left of red block and move it to area d
left_block_name = parse_obj_names('the block left of red block', f'objects = {get_obj_names()}', f'objects = {get_obj_names()}', f'objects = {get_obj_names()}')
if left_block_name:
    say('Picking up the (left_block_name) to area d')
    put_first_on_second(left_block_name, parse_position('area d'))
else:
    say('There is no block left of the red block')

LMP parse_obj_names exec:
objects = ['blue block', 'green block', 'red block']
# the block left of red block
block_names = ['blue block', 'green block', 'red block']
block_positions = get_obj_positions_by(block_names)
red_block_idx = block_names.index('red block')
left_block_idx = red_block_idx - 1
left_block_name = block_names[left_block_idx]
ret_val = left_block_name

robot says: Picking up the green block to area d
LMP parse_position exec:
# area d
ret_val = get_obj_pos('area d')

[INFO] [177754324.557454]: Executing action via /sgr532/sgr_ctrl | object=green block target=None pick=[0.20170546 0.10150751 0.02 ]
place=[0.26 0.25 0.02]

```

(a) Generated policy for relative-position reasoning



(b) Real robot moving the inferred target block to area D

Fig. 7. Relative-position reasoning demonstration. The LMP identifies the block to the left of the red block, parses area d, and generates the corresponding pick-and-place action for real-robot execution. The real-robot image shows the inferred target block being moved toward the D region.

camera-robot calibration, to reduce the influence of environmental variation on grasp accuracy.

Third, the current `exec_safe` provides only minimal safety checks. It mainly prevents import, double underscores, and direct `exec/eval`. It should not be considered

equivalent to a fully isolated sandbox. For long-term open interaction, stricter AST whitelisting, API permission control, and action-boundary validation should be added.

Finally, the current self-correction mainly handles `grasp_failed`. The system has not yet implemented LMP-

Author Contributions

This project was completed collaboratively by all team members. The individual contributions are summarized below.

Team Member	Contributions
Yibai Sun and Peiming Chen	Contributed to the overall project design and technical planning, including solution discussion, system implementation, real-robot deployment, code writing, and code testing. They also took major responsibility for writing the final report and preparing both the midterm and final presentations.
Rui He, Yinjie Liao, and Jiaxi Zhong	Participated in the initial project discussion and early-stage planning. They were mainly responsible for writing Lab 1 and Lab 2, and also contributed to partial code testing during the project development process.